Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

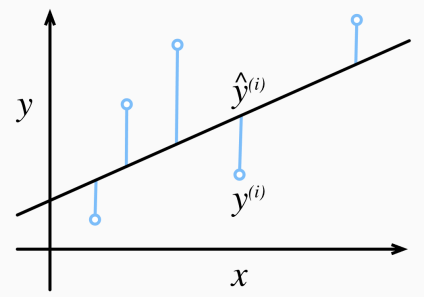
Deep Learning Architectures 1: MLPs

From Linear Regression as DL

- Recall our simple linear regression model

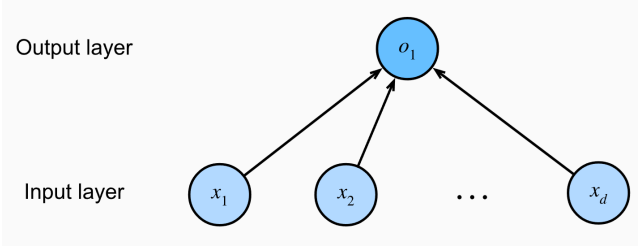
$$y_i = x_i^\top w^* + b^* + e_i \tag{1}$$

- labels $y_i \in \mathbb{R}$, features $x_i \in \mathbb{R}^d$, noise $e_i \in \mathbb{R}$
- regression weights $w^* \in \mathbb{R}^d$, bias $b^* \in \mathbb{R}$
- Previously we ignored the bias term. That is w.l.o.g. as we can increase the dimension to $d+1$ by defining $w^* \leftarrow (w^*, b^*)$ and $x_i \leftarrow (x_i, 1)$



From Linear Regression as DL (Cont'd)

- We can think of this model as a combination of artificial neurons: elements taking an input signal processing it, and pass it forward
- That is, a simple Artificial neural network (ANN) or a shallow DL model
- The training involves fitting the parameters of the neurons, i.e., the regression parameters, to the data





- Can also be thought as maximum likelihood estimation when the noise is Gaussian and observations are statistically independent

$$\max_{w,b} p(y|x) := \prod_{i=1}^{n} p(y_i|x_i) \propto \prod_{i=1}^{n} \exp(-\frac{(y_i - x_i^\top w - b)^2}{2\sigma^2}) \tag{2}$$

- As the log is a monotone function, the above is equivalent to the following negative log likelihood minimization problem

$$\min_{w,b} \sum_{i=1}^{n} (y_i - x_i^\top w - b)^2 \tag{3}$$

- Most advance learning tasks follow a similar template: maximize the likelihood while assuming suitable distributional assumptions.
- Some times we also include a prior (aka a regularization term) and the problem becomes Maximum a priori estimation (MAP)

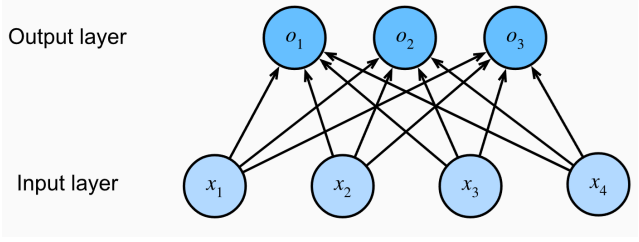
Classification as DL

- Now assume a classification problem where instead $y_i \in \{1, \dots, C\}$
- We can represent the labels via one-hot encoding:

$$y_i \in \{e_1, \dots, e_C\}, e_i \in \mathbb{R}^C \qquad e_i = (0, 0, \dots, 1, 0, 0) \tag{4}$$

  That is, e_i's are the standard unit basis vectors in $\mathbb{R}^C$.

- Again a simple linear relation between features and labels: $y_i = W^* x_i + b^*$ where $y_i \in \mathbb{R}^C$ and $x_i \in R^d$ are labels and features while $W^* \in \mathbb{R}^{C \times d}$ and $b^* \in \mathbb{R}^C$ are weights and bias





- We can use square loss as before, but the solution is not elegant
- Let $\hat{W}$ and $\hat{b}$ denote the parameters of a learned model.
- Consider the predicted label $\hat{y}_i = \hat{W}x_i + \hat{b}$
- Desirable to think of the c's entry of $\hat{y}_i$ as the probability that the label of x is c.
- Thus, we want $\hat{y}_i$ (like the true labels y_i after one-hot encoding) to be nonnegative and sum to one, i.e., be probability vectors in $\mathbb{R}^C$
- We perform Softmax normalization

$$\hat{o} = \hat{W}x + \hat{b}, \qquad \hat{y} = \mathrm{softmax}(\hat{o}), \qquad \hat{y}(j) := \frac{exp(\kappa \hat{o}(j))}{\sum_{c=1}^{C} exp(\kappa \hat{o}(c))} \tag{5}$$

- Motivated by the Boltzmann distribution in Physics to model a distribution over energy states in gas molecules (in our case the states are the finite classes).

Classification and MLE (Cont'd)



- The training involves fitting the parameters of the neurons, i.e., the classification parameters, to the data. But which loss?
- Let us resort to MLE: find the parameters to maximize the likelihood of data
- Given our softmax normalization, the natural observation/noise distribution is multinomial (discrete Boltzmann) distribution

$$\begin{aligned} \hat{y}(j) = p(j|x) &= \frac{exp(\hat{o}(j))}{\sum_{c=1}^{C} exp(\hat{o}(c))} \quad \text{probability of class/state } j \text{ given feature } x \\ -\log p(j|x) &= -\log \hat{y}(j) = -\sum_{c=1}^{C} y(j) \log \hat{y}(j) \quad \text{only one coordinate of } y(j) \text{ is nonzero} \\ &= \text{Cross-Entropy}(y, \hat{y}) \end{aligned} \tag{6}$$

- A notion of distance between two distributions and related to the Kullback–Leibler (KL) divergence

Entropy, KL, and Cross-Entropy



- Entropy: how random a quantity is (more random, more bits needed to represent it)

$$H(p) = -\sum_{c=1}^{C} p(c) \log p(c) \geq 0 \tag{7}$$

- KL divergence: how different two distributions are

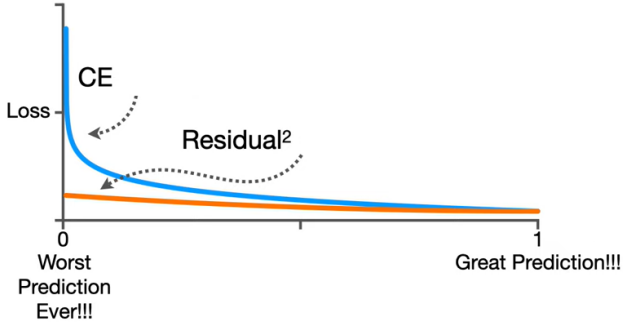
$$D_{kl}(p\|q) = \sum_{c=1}^{C} p(c) \log \frac{p(c)}{q(c)} \geq 0 \tag{8}$$

  An example of f-divergence with convex functions f: $D_f(p\|q) = \mathbb{E}_q[f(\frac{p}{q})]$. For KL, $f(t) = t \log t$.

- Easy to show Cross-Entropy$(y, \hat{y}) = H(y) + D_{kl}(y\|\hat{y}) \geq 0$.
- Thus, minimizing CE loss is equivalent to minimizing KL divergence.

CE vs Square loss

- As discussed CE is more natural than square loss as it enables us to think of lables as probabilities. A direct link to MLE
- CE provides larger feedback sooner





- Consider linear regression, i.e., $\hat{y} = w^\top x$. For training we use SGD to minimize ERM with square loss. By chain rule, Gradient is proportional to the gradient w.r.t. the output $\hat{y}$:

$$\ell(w,z) = \frac{1}{2}(y-\hat{y})^2, \qquad \nabla_w \ell(w,z) = -x(y - w^\top x) \propto \nabla_{\hat{y}} \ell(w,z) = \hat{y} - y \tag{9}$$

- Consider softmax classification, i.e., $\hat{y}(j) := \frac{exp(\hat{o}(j))}{\sum_{c=1}^{C} exp(\hat{o}(c))}$ where $\hat{o} = Wx$. For training we use SGD to minimize ERM with CE loss. By chain rule, Gradient is proportional to the gradient w.r.t. the output $\hat{y}$

$$\ell(w,z) = CE(y,\hat{y}) = -\sum_{c=1}^{C} y(c) \log \hat{y}(c), \qquad \nabla_{\hat{y}} \ell(w,z) = \mathrm{softmax}(\hat{o}) - y = \hat{y} - y \tag{10}$$

- In both cases, the larger the error/residual, the larger the gradient/signal



- Let us explore why we need hidden layers.
- Binary classification with hinge loss

$$y = \text{sign}(\langle x, w \rangle), \qquad \ell(w, z) = \max(0, 1 - y\langle x, w \rangle) \tag{11}$$

- Consider SGD for this problem.
- For $t = 1, \ldots, T$
  - sample $z_t = (x_t, y_t) \sim p_z$
  - If $y_t \langle x_t, w_t \rangle < 0$ perform $w_{t+1} = w_t + \eta y_t x_t$
  - If $y_t \langle x_t, w_t \rangle \geq 0$ perform $w_{t+1} = w_t$
- This is called Perceptron and given a margin $\rho > 0$ between classes has $\mathcal{O}(\rho^{-2})$ guaranteed on the number of mistakes
- What if data non-linearly separable?

Perceptron (Cont'd)

- Assume we have two features $x = (x^1, x^2)$ and $y = \text{xor}(x^1, x^2)$
- data non-linearly separable and Perceptron fails
- Idea: combine multiple Perceptrons

$$o_3 := \text{xor}(x^1, x^2) = x^1 \cdot \bar{x}^2 + x^2 \cdot \bar{x}^1 = (x^1 + x^2) \cdot (\bar{x}^1 + \bar{x}^2) := o_1 \cdot o_2 \tag{12}$$

Symbol	A	B	Q
	0	0	0
	1	0	1
	0	1	1
	1	1	0
Q = A ⊕ B			

Perceptron (Cont'd)



- $o_1 = x^1 + x^2$

x^1	1	0	1	0
x^2	1	0	0	1
o_1	1	0	1	1

- $o_2 = \bar{x}^1 + \bar{x}^2$

$\bar{x}^1$	1	0	1	0
$\bar{x}^2$	1	0	0	1
o_2	0	1	1	1

- $o_3 = o_1 \cdot o_2$

o_1	1	0	1	0
o_2	1	0	0	1
o_3	1	0	0	0

Multi-Layer Perceptron (MLP)



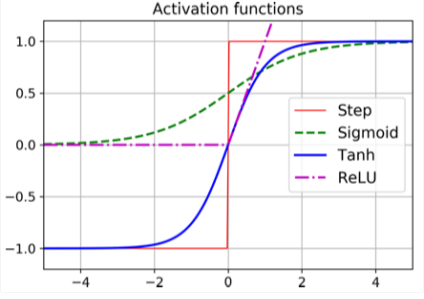
- Each rule can be learned by a single Perceptron. To be thought as a neuron with sign as the activation

$$\text{output} = \text{sign}(\text{input} \times \text{weight}) \approx \text{ReLU}(\text{input} \times \text{weight}) = \max(0, \text{input} \times \text{weight}) \tag{13}$$

- Thus, to learn the final label rule we are building a Multi-Layer Perceptron (MLP) neural network
- MLP enjoys a higher learnability capacity (characterized by VC dimension) than a single Perceptron, thus able to handle complex data
- VC dim: Size of the largest dataset of points in general positions s.t. we can classify it regardless of label assignment
- VCdim of percepton is roughly d but for MLP it is roughly Ld when having L hidden layers.

Activations

- Sigmoid: $\sigma(x) = \frac{1}{1+\exp(-x)}$
- Tanh: $\sigma(x) = \frac{1-\exp(-2x)}{1+\exp(-2x)}$
- ReLU: $\sigma(x) = \max(0, x)$
- ReLU leads to more efficiency in computation and stability in training
- Activations are applied coordinate-wise



Why nonlinear Activations?



- Two hidden layers in an MLP with linear activations

$$h_1 = W_1 x + b_1, \qquad h_2 = W_2 h_1 + b_2 \tag{14}$$

- These hidden layers will collapse into one

$$\tilde{h} = \tilde{W} x + \tilde{b}, \qquad \tilde{W} = W_2 W_1, \qquad \tilde{b} = W_2 b_1 + b_2 \tag{15}$$

- Thus, intuitively, we have lost the increase in the learning capacity we gained by having hidden layers